

Mapeo de relaciones con JPA

Continuando con la implementación de Hibernate en nuestro proyecto, vimos como mapear relaciones entre objetos.

Para esto creamos una nueva entidad, **Issue** que representa un issue que debe ser corregido, y lo mapeamos:

```
@Entity
public class Issue {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "issue_id_seq")
    @SequenceGenerator(sequenceName = "issue_id_seq", name = "issue_id_seq", allocationSize = 1)
    private Long id;

    @Column(length = 1024, nullable = false)
    private String description;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    private User createdBy;

    @ManyToOne(fetch = FetchType.EAGER, optional = true)
    private User assignedTo;
```

```
@Enumerated(EnumType.STRING)
```

```
private Priority priority;
```

```
/* default */ public Issue() {
```

```
    // Just for Hibernate
```

```
}
```

```
public Issue(final String description, final User createdBy,
```

```
            final Priority priority) {
```

```
    super();
```

```
    this.description = description;
```

```
    this.createdBy = createdBy;
```

```
    this.priority = priority;
```

```
}
```

```
public User getAssignedTo() {
```

```
    return assignedTo;
```

```
}
```

```
public void setAssignedTo(final User assignedTo) {
```

```
    this.assignedTo = assignedTo;
```

```
}
```

```
public Priority getPriority() {
```

```
    return priority;
```

```
}
```

```
public void setPriority(final Priority priority) {  
    this.priority = priority;  
}
```

```
public void setCreatedBy(User createdBy) {  
    this.createdBy = createdBy;  
}
```

```
public User getCreatedBy() {  
    return createdBy;  
}
```

```
public Long getId() {  
    return id;  
}
```

```
public String getDescription() {  
    return description;  
}
```

```
@Override
```

```
public String toString() {  
    return "IssueTPE [id=" + id + ", description=" +  
description  
        + ", createdBy=" + createdBy.getUsername(  
)  
        + ", assignedTo=" + (assignedTo == null ?  
"null" : assignedTo.getUsername())
```

```
        + ", priority=" + priority + "]);  
    }  
}
```

Definimos que la relación con **User** es opcional para la asignación, ya que no es una relación fuerte (pueden existir issues que no fueron asignados a nadie); pero es requerida para la creación (alguien tuvo que haberlos creado).

Por su parte, **Priority** es un enum standard de java:

```
public enum Priority {  
    LOW,  
    MEDIUM,  
    HIGH,  
    UNBREAK_NOW,  
}
```

Como vimos en clase **@Enumerated** tiene varias opciones para mapear enums, según que querramos priorizar. En general, **EnumType.STRING** es una buena opción, pues favorece la legibilidad de los datos en la base de datos (concepto que ya manejan de bases de datos 1). Se debe tener cuidado si se agregan nuevos valores al enum en garantizar que sus nombres entren en el ancho de columna que haya generado Hibernate.

Por su parte, desde **User** mapeamos las relaciones en forma inversa:

```
@OneToMany(fetch = FetchType.LAZY, orphanRemoval = false, mappedBy = "assignedTo")
private List<Issue> assignedIssues;

@OneToMany(fetch = FetchType.LAZY, orphanRemoval = true, mappedBy = "createdBy")
private List<Issue> createdIssues;
```

Este annotation tiene un atributo, `fetchType` que nos permite jugar con cuando y como queremos obtener los objetos que participan de la relación. Analizando los logs vemos los peligros de estos atributos, ya que de forma inadvertida estabamos trayendo todos los registros de usuarios e issues asignados de la base de datos. JPA nos da un enorme poder para simplificar el código de la aplicación, pero no debemos caer nunca en desentendernos de la base de datos, y tenemos que auditar cuidadosamente que queries estamos haciendo al impactar cambios a nuestros modelos / mapeos. Incluso cuando localmente todo funciona bien, al salir a producción, con miles / millones de registros, este tipo de problemas pueden rápidamente deteriorar la performance de la aplicación y hasta hacer caer el servicio.

Finalmente, vimos otra gran diferencia entre contar con una conexión y contar con un contexto de persistencia. Implementamos un método en la capa de servicio para ordenar los issues de un usuario por prioridad:

```
@Transactional
```

```

@Override
public void orderIssuesByPriority(final long id) {
    final User user = userDao.findById(id);
    Collections.sort(user.getPendingFixes(), new Comp
arator<IssueTPE>() {

        @Override
        public int compare(final IssueTPE o1, final I
ssueTPE o2) {
            return o2.getPriority().ordinal() - o1.ge
tPriority().ordinal();
        }
    });
}

```

Si bien nunca hacemos una llamada al DAO para persistir los cambios al orden de la lista, dado que el usuario es una instancia manejada por el contexto de persistencia (fué creada por el entity manager dentro de un mismo contexto transaccional), se da cuenta dicho atributo está “dirty” y va a intentar persistirlo al salir. De nuevo, tenemos gran poder para reducir código, pero:

1. nos atamos a usar JPA / Hibernate, volver a JDBC implica perder la persistencia sin que el código deje de compilar.
2. tenemos poco control de cuando se están ejecutando las queries y cuales. Si usasemos un filtro para tener una sesión abierta en la vista, esto no ocurriría hasta que no se termine de

renderizar la respuesta

Notese que esto funciona porque el `User` es creado por el contexto de persistencia actual. Si `User` es pasado por argumento, y no tenemos un filtro / otro método alrededor que nos mantenga dentro de un mismo contexto, esto no ocurriría, por el contrario,

`user.getPendingFixes()` podría tirar una excepción al querer hacer lazy loading de la colección sin tener una sesión asociada. El

`EntityManager` ofrece métodos para reasociar una instancia al mismo, pero exponer estos métodos al servicio lo ensucian y exponen detalles de implementación del DAO (el uso de JPA / Hibernate).

Otro punto interesante sobre este método es analizar lo que está haciendo Hibernate para persistir el cambio de orden de la colección. Vemos que borra todas las relaciones entre el usuario y todos sus issues y vuelve a crearlos en el orden apropiado. Esto se debe a que están usando el `ROWID` de la tabla relación para usar el orden, y necesita que el orden de inserción sea el orden en que queremos recuperarlos.